

Laboratoire de High Performance Coding

semestre printemps 2026

Laboratoire 7 : Parallélisme des tâches

Temps à disposition : 6 périodes (3 séances de laboratoire).

1 Introduction

Ce laboratoire a pour objectif de vous familiariser avec les techniques et outils de parallélisation des tâches. Vous apprendrez à implémenter des solutions multithreadées pour des problèmes de traitement de données.

2 Calcul multithreadé de la fréquence de k-mers dans un fichier texte

Dans cette activité, vous devez analyser un fichier texte (par exemple contenant les chiffres de Pi, de l'ADN ou tout autre texte brut volumineux) afin de calculer la fréquence de toutes les sous-séquences (« k-mers ») de taille k , où k est fourni en paramètre. Par exemple, pour $k = 3$ sur la chaîne ABCDABC, les k-mers extraits sont : 2xABC, BCD, CDA, DAB.

2.1 Objectifs

- Améliorer la logique et le traitement de la version mono-threadée fournie.
- Implémenter une version multithreadée (OpenMP ou pthreads).
- Comparer les performances entre votre version multithreadée et la version mono-threadée améliorée.

Le code fourni lit naïvement le fichier donné et compte le nombre d'occurrences de chaque k-mer de longueur k . Vous êtes libres de le modifier, d'ajouter ou de supprimer ce que vous souhaitez. Veuillez simplement à ne pas modifier le comportement du programme ni son interface en ligne de commande.

Vous devrez également modifier le `CMakeLists.txt` en conséquence.

Voici quelques fichiers volumineux contenant les chiffres de Pi que vous pouvez utiliser pour tester votre solution :

- <https://pi2e.ch/blog/2017/03/10/pi-digits-download/#download>
- <https://calculat.io/en/number/download-pi-digits-files-txt-zip>

Vous êtes toutefois libres d'utiliser n'importe quel fichier texte de votre choix comme entrée.

3 Pan-Tompkins

Pour conclure ce laboratoire, nous vous invitons à revenir une dernière fois sur le code développé durant le lab01.

Nous vous demandons d'adapter votre programme afin qu'il lise un flux continu de données. Il ne devra donc plus traiter un ensemble complet de données, mais fonctionner par paquets.

L'objectif de cette étude de cas est également de rendre la parallélisation intéressante. Afin d'éviter de perdre certaines informations du signal (par exemple un pic situé entre deux paquets), nous vous demandons d'utiliser un chevauchement (*overlapping*) entre les paquets.

L'idée est de traiter des paquets de 1000 échantillons avec un chevauchement de 250 échantillons. Vous êtes libres de choisir la manière dont vous souhaitez organiser cela.

4 Travail à rendre

- Deux programmes fonctionnels et compilables :
 - Un pour l'activité sur les k-mers.
 - Un pour l'activité Pan-Tompkins.
- Un rapport structuré en deux parties :
 - **Première partie — Analyse des k-mers :**
 - * Une explication des éléments inefficaces du code fourni, ainsi que des améliorations apportées.
 - * Une analyse des performances de votre version mono-threadée.
 - * Une description de la stratégie de parallélisation mise en œuvre : répartition du travail entre les threads, traitement des cas limites, zones de chevauchement, etc.
 - * Une comparaison détaillée entre les performances des versions mono-threadée et multithreadée (temps d'exécution, scalabilité, goulots d'étranglement, etc.).
 - **Deuxième partie — Activité Pan-Tompkins :**
 - * Une description de la partie de votre code qui a été parallélisée.
 - * Une explication claire de la stratégie de parallélisation utilisée.
 - * Une justification des choix effectués, ainsi qu'une évaluation de l'intérêt et de l'efficacité de votre parallélisation.

Information

N'hésitez pas à modifier le `CMakeLists.txt` afin de générer directement deux exécutables : une version mono-threadée et une version multithreadée.